

PTO 与 NVIDIA TileIR

技术对比

程序员风格技术文档

覆盖 PTO-AS / PTO-BC / PTO ISA 与 TileIR

版本: 3.0

日期: 2026年2月

一句话总结

目录

1.1 编译流水线对比

PTO 流水线

TileIR 流水线

1.2 层级对应

2.1 Tile 是什么？

PTO 的 Tile (tile_buf)

TileIR 的 Tile

2.2 View 和 Partition

PTO

2.3 内存模型

TileIR：令牌系统 (TKO)

PTO：事件 + TSYNC

TileIR

PTO

TileIR

PTO

PTO 优势

TileIR 优势

>

PTO 和 TileIR 都是为了解决同一个问题：让程序员能方便地写高性能矩阵运算，同时又能充分利用硬件能力。

简单说： – TileIR = 宜家模式 —— 给你一套标准化"料理台" (Tile)，你把食材放上去，系统自动帮你烹饪 – PTO = 木匠模式 —— 告诉木匠你想要什么样的家具，他来决定用什么木材、怎么切割

1. 整体架构

1.1 编译流水线对比

PTO 流水线



TileIR 流水线



1.2 层级对应

PTO	TileIR	说明
pyPTO	CuTile	Python 前端
PTOAS	cuda_tile	核心 IR
PTOBC	TileIR bytecode	字节码
PTO ISA	nv_tileaa→nv_tileas→SASS	目标代码

2. 核心概念

2.1 Tile 是什么？

Tile = 二维数据块。把大矩阵想象成一张照片，Tile 就是把照片切成的小方块。每个小方块独立处理，最后拼起来就是完整结果。

PTO 的 Tile (tile_buf)

```
!pto.tile_buf<
  loc=Acc,           // 位置: Left=左矩阵, Right=右矩阵, Acc=累加器
  dtype=f16,         // 数据类型
  rows=256, cols=16, // 物理容量
  v_row=dyn, v_col=16, // 有效区域 (支持动态)
  blayout=RowMajor,  // Buffer 布局
  fractal=512        // 分形维度 (Tensor Core 用)
>
```

关键点： – `loc` (位置) 是 PTO 独创 —— 告诉硬件这个 Tile 充当什么角色 (Left/Right/Acc)，硬件可以据此优化 – `v_row/v_col` 是有效区域 —— 物理容量可能很大，但这次计算只用到一部分

TileIR 的 Tile

```
!cuda_tile.tile<16x16xf32>
```

相对简单，布局通过属性表达。

2.2 View 和 Partition

比喻：从书架拿书 – **GlobalTensor** = 整个书架 – **Partition View** = 要拿的第3–5层 – **Tile** = 拿在手里的书

PTO

```
// 1. 建立全局视图 (声明书架)
%view = pto.make_tensor_view %ptr,
  dtype=f32,
  shape=[1,1,16,1024,1024],
  strides=[1048576,1048576,1048576,1024,1]
: !pto.tensor_view<...>

// 2. 切分窗口 (决定拿哪几层)
%p = pto.partition_view %x,
  offsets=[0,0,0,0,0],
  sizes=[1,1,16,16,16]
: !pto.tensor_view<...> -> !pto.partition_tensor_view<...>

// 3. 加载到 Tile
pto.tload ins(%p : !) outs(%tile : !)
```

2.3 内存模型

TileIR: 令牌系统 (TKO)

洗菜的要给切菜的一个"令牌", 切菜的拿到令牌才能开始。

```
// 加载 → 产生令牌
%tile0, %token0 = cuda_tile.load_view_tko weak %view[...] : ... -> token

// 计算 → 消耗令牌
%result = cuda_tile.mmaf %tile0, %tile1, %acc : ...

// 存储 → 需要令牌
%token = cuda_tile.store_view_tko weak %result, %out[...] : ... -> token
```

令牌类型: `weak`、`relaxed`、`acquire`、`release`、`acq_rel`

PTO: 事件 + TSYNC

更像餐馆的"叫号"系统 —— 做完一道菜喊"好了!", 下一个人听到就开始。

```
// 加载 (自动产生事件)
pto.tload ins(%view0 : !) outs(%tile0 : !)

// 计算 (依赖前面的事件)
"pto.tmatmul"(%tile0, %tile1, %acc) : ...

// 同步点
pto.tsync %acc

// 存储 (等同步点完成)
pto.tstore ins(%acc : !) outs(%out : !)
```

3. 操作映射

TileIR	PTO	说明
<code>cuda_tile.load_view_tko</code>	<code>pto.tload</code>	加载
<code>cuda_tile.store_view_tko</code>	<code>pto.tstore</code>	存储
<code>cuda_tile.gather</code>	<code>pto.mgather</code>	聚集
<code>cuda_tile.scatter</code>	<code>pto.mscatter</code>	散射
<code>cuda_tile.mmaf</code>	<code>pto.tmatmul</code>	矩阵乘
<code>cuda_tile.addf</code>	<code>pto.tadd</code>	加法

TileIR	PTO	说明
<code>cuda_tile.make_tensor_view</code>	<code>pto.make_tensor_view</code>	创建视图
<code>cuda_tile.partition</code>	<code>pto.partition_view</code>	分区
<code>cuda_tile.get_tile_block_id</code>	<code>pto.get_block_idx</code>	获取 ID
<code>pto.rowsum</code>	—	行求和
<code>pto.colsum</code>	—	列求和

4. GEMM 矩阵乘法

TileIR

```
%c = cuda_tile.mmaf %a, %b, %c
{precision = fp32, rounding = dynamic}
: (tile<16x16xf16>, tile<16x16xf16>) -> tile<16x16xf32>
```

精度：`fp32`、`fp16`、`bf16`、`tf32`、`int8`、`int4`

PTO

```
func @gemm(%t1: !pto.tile_view<16x16xf16>,
           %t2: !pto.tile_view<16x16xf16>,
           %t3: !pto.tile_view<16x16xf32>) {

  // 声明 Tile (关键: 显式 location)
  %a_left = pto.alloc_tile() : !pto.tile_buf<loc=left, dtype=f16>
  %b_right = pto.alloc_tile() : !pto.tile_buf<loc=right, dtype=f16>
  %c_acc = pto.alloc_tile() : !pto.tile_buf<loc=acc, dtype=f32>

  // 加载
  pto.tload ins(%t1 : !) outs(%a_left : !)
  pto.tload ins(%t2 : !) outs(%b_right : !)

  // 矩阵乘
  "pto.tmatmul"(%a_left, %b_right, %c_acc) : ...

  // 存储
  pto.tstore ins(%c_acc : !) outs(%t3 : !)
}
```

5. 控制流

TileIR

```
cuda_tile.for %i = 0 to 16 { ... }
cuda_tile.while %i < %N { ... }
cuda_tile.if %cond { ... } else { ... }
```

PTO

前端静态展开，IR 层不提供显式控制流：

```
for i in range(16):
    tile = pto.tload(view[i])
    result = pto.tadd(tile, tile)
    pto.tstore(out[i], result)
```

6. 字节码对比

项目	PTO-BC v0	TileIR
Magic	PT0BC\0 (6字节)	\x7FTileIR\x00 (8字节)
版本	u16	major:minor:tag
VarInt	LEB128	PrefixVarInt

7. 设计差异

方面	PTO	TileIR
Tile	显式 location	抽象 shape
布局	编译时确定	运行时推导
同步	event + TSYNC	token
后端	多硬件支持	NVIDIA 专用
控制流	前端展开	IR 层支持

PTO 优势

- 一套代码 → NPU + GPU
- location 模型让硬件更好优化
- 虚拟 ISA，硬件升级不用改上层

TileIR 优势

- 更丰富的数学函数
- 精细的精度控制
- NVIDIA 生态完整

8. 决策建议

- 要跨后端：选 PTO
 - 要 NVIDIA 深度优化：选 TileIR
-

版本：3.0 2026年2月